Pokerbots

Protocol Specification

Discard Hold'em — stdin/stdout interface

Submission deadline: 15 November 2026, 23:59 IST

This specification is part of the Pokerbots Rulebook. Values marked **(provisional)** may be adjusted after benchmarking. Corrections or clarifications to this specification are announced to all teams.

1 Overview

A bot is a Python 3.11 program (`bot.py`) started fresh for each leg of every match. A match is played in two legs; each leg contains every deal of the match once, and in the second leg the hole cards and seats are swapped between the bots (Rulebook 7.1). The bot communicates with the match engine *only* through **stdin** (messages from the engine) and **stdout** (the bot's replies). There is no network and no other channel. The engine is authoritative: it deals the cards, tracks the game state, enforces legality and runs the clock.

The game is heads-up no-limit Hold'em with two changes: each player receives **three** hole cards, and after the flop betting each player **discards one** hole card, simultaneously and face up. The game rules are in Section 4 of the Rulebook. The Problem Statement gives them in full with worked examples.

2 Transport and Format

- **Encoding:** UTF-8. One message per line, terminated by `\n`.
- **Format:** each line is a single JSON object with a `"type"` field.
- **Direction:** the engine writes to the bot's stdin; the bot writes to its stdout.
- **Flushing:** the bot must flush stdout after every message (`print(..., flush=True)`). Unflushed output does not reach the engine and consumes time bank.
- **stdout** must contain only protocol messages. A line sent while a request is outstanding is taken as the reply, so a debug print is a malformed reply (Section 9) and the real reply after it is ignored.
- **stderr** may be used for debugging. It is captured, limited to 64 KB per leg, and never parsed.
- The bot reads one line at a time and replies *only* to a request (`action_request` or `discard_request`). A reply sent when no request is outstanding is ignored.
- **Forward compatibility:** the bot must ignore message types and fields it does not recognise.

3 Conventions

- **Cards** are two-character strings: rank (`2 3 4 5 6 7 8 9 T J Q K A`) then suit (`s h d c`), for example `As`, `Td`, `2c`. Ace is high in rank comparisons.

- **Chips** are integers. Blinds are 1 and 2. Every hand starts with 200 chips per player (stacks reset every hand).
- **Seats** are `"button"` (posts the small blind) and `"big_blind"`. Seats alternate every hand.
- **Streets** are `"preflop"`, `"flop"`, `"turn"` and `"river"`.
- **Actors** in history entries are `"me"` (the receiving bot) and `"opp"`.
- **Amounts** for `call` and `raise` mean the *total chips the actor has committed on the current street after the action* (a "to" amount), not the increment.
- The bot is not told which deals are duplicates of each other. It may keep state between hands within a leg, but it keeps no state between legs or between matches.

4 Message Flow

The flow below is one leg. A match consists of two legs, and each leg is played by a newly started bot process, which receives its own `init` and `match_end`.

```
engine -> bot init (once per leg)
repeat for each hand:
  engine -> bot new_hand
  engine -> bot action_request (preflop, each time the bot must act)
  bot -> engine action
  ... flop betting ...
  engine -> bot discard_request (once, only if nobody has folded)
  bot -> engine discard (both players answer simultaneously)
  engine -> bot discard_result
  ... turn and river betting ...
  engine -> bot hand_result
engine -> bot match_end (once per leg; the bot should then exit)
```

A hand can end early: if a player folds, the engine sends `hand_result` immediately and no further requests follow. If a player folds before the flop betting ends, there is no discard. If both players are all-in, the remaining board cards are dealt and the discard still takes place after the flop.

5 Engine → Bot Messages

5.1 `init` (once per leg)

Field	Type	Meaning
`type`	str	`"init"`
`protocol_version`	str	`"1.0"`
`total_hands`	int	number of hands in this leg (200 in Stage 1, 500 in Stage 2)
`starting_stack`	int	200
`small_blind`, `big_blind`	int	1 and 2
`time_bank_ms`	int	total time bank for this leg **(provisional)**
`per_response_ms`	int	limit for any single reply, 5000 **(provisional)**

5.2 `new_hand`

Field	Type	Meaning
`type`	str	`"new_hand"`
`hand_id`	int	1-indexed within the leg
`seat`	str	`"button"` or `"big_blind"`
`hole_cards`	list[str]	your 3 hole cards
`stack`	int	your stack before blinds (200)
`opp_stack`	int	opponent's stack before blinds (200)

5.3 `action_request`

The bot must answer with an `action` message.

Field	Type	Meaning
`type`	str	`"action_request"`
`hand_id`	int	current hand
`street`	str	current street
`board`	list[str]	community cards dealt so far
`pot`	int	total chips in the pot, including bets on this street
`my_bet`, `opp_bet`	int	chips each player has committed on this street
`to_call`	int	`opp_bet` − `my_bet`; 0 means checking is legal
`my_stack`, `opp_stack`	int	chips each player still has behind
`legal_actions`	list[str]	subset of `fold`, `check`, `call`, `raise`
`min_raise`, `max_raise`	int	legal range of raise-to amounts; present only if `raise` is legal
`raises_left`	int	bets or raises still allowed on this street
`history`	list[object]	all voluntary actions this hand, oldest first (blinds are not listed)
`time_left_ms`	int	remaining time bank for this leg

A history entry has the form `{"street":"flop","actor":"opp","action":"raise","amount":6}`. `amount` is present for `call` and `raise` only.

5.4 `discard_request`

Sent to both players at the same time after the flop betting ends. The bot must answer with a `discard` message.

Field	Type	Meaning
`type`	str	`"discard_request"`
`hand_id`	int	current hand
`hole_cards`	list[str]	your 3 hole cards
`board`	list[str]	the 3 flop cards
`pot`	int	total chips in the pot
`my_stack`, `opp_stack`	int	chips each player still has behind
`time_left_ms`	int	remaining time bank for this leg

5.5 `discard_result`

Sent after both discards are in. Both discarded cards are now public.

Field	Type	Meaning
`type`	str	`"discard_result"`
`hand_id`	int	current hand
`my_discard`	str	the card you discarded
`opp_discard`	str	the card the opponent discarded
`hole_cards`	list[str]	your 2 remaining hole cards

5.6 `hand_result`

Field	Type	Meaning
`type`	str	`"hand_result"`
`hand_id`	int	hand number
`result`	str	`"fold"` or `"showdown"`
`net`	int	chips you won (positive) or lost (negative) this hand
`board`	list[str]	board cards dealt
`my_hole_cards`	list[str]	your remaining hole cards (3 if the hand ended before the discard)
`opp_hole_cards`	list[str] or null	the opponent's 2 remaining cards, only at showdown
`opp_discard`	str or null	the opponent's discard, if a discard took place
`history`	list[object]	complete action history of the hand

5.7 `notice`

Informational only; no reply is expected.

Field	Type	Meaning
`type`	str	`"notice"`
`code`	str	`"malformed"`, `"illegal_action"` or `"illegal_discard"`
`applied`	str	what the engine did instead, e.g. `"check"`, `"fold"` or a card

5.8 `match_end`

Sent once at the end of each leg.

Field	Type	Meaning
`type`	str	`"match_end"`
`reason`	str	`"complete"`, `"opp_forfeit"` or `"forfeit"`

After `match_end` the bot should exit. A `reason` of `"forfeit"` or `"opp_forfeit"` in the first leg means the second leg will not be played.

6 Bot → Engine Messages

6.1 `action`

Reply to each `action_request` with exactly one line:

```
{"type":"action","action":"fold"}
{"type":"action","action":"check"}
{"type":"action","action":"call"}
{"type":"action","action":"raise","amount":40}
```

6.2 `discard`

Reply to each `discard_request` with exactly one line naming one of your 3 hole cards:

```
{"type":"discard","card":"Kd"}
```

7 Legality Rules

- `action` must be one of `legal_actions`.
- `fold` is legal only when `to_call` > 0.
- `check` is legal only when `to_call` $= 0$.
- `call` is legal only when `to_call` > 0. It matches the opponent's bet.
- `raise` (which also covers an opening bet) needs an integer `amount` with `min_raise` $\leq$ `amount` $\leq$ `max_raise`.
- `min_raise` = `opp_bet` + $\max(s, 2)$, where s is the size of the last bet or raise on this street. With no bet yet the minimum is 2. The big blind counts as the first raise preflop, so the first preflop minimum is 4.
- `max_raise` is the all-in amount: the most either player can have committed on this street. If it is below the usual minimum, `min_raise` equals `max_raise` and only that all-in raise is legal.
- At most 4 bets or raises are allowed per street (the big blind is the first one preflop). When `raises_left` is 0, `raise` is not in `legal_actions`.
- `raise` can also be absent while `raises_left` > 0, when the opponent is all-in. Always choose from `legal_actions`.
- A `discard` card must be one of the bot's current 3 hole cards.

8 Time Limits (provisional)

- **Time bank:** one cumulative bank per leg, not reset between hands: 30 seconds per leg in Stage 1 (two legs of 200 hands), 75 seconds per leg in Stage 2 (two legs of 500 hands).
- **What is charged:** the time from when the engine sends an `action_request` or `discard_request` until it receives the complete reply line. Both discard clocks run at the same time.
- **Per reply:** any single reply must arrive within 5 seconds.
- **Start-up:** 5 seconds at the start of each leg. The clock for each request starts at the later of the moment it is sent and 5 seconds after launch, so time before that point is not charged, and the first reply may arrive up to 10 seconds after launch.
- **Wall-clock kill:** 90 seconds per leg in Stage 1 and 225 seconds per leg in Stage 2.
- `time_left_ms` is reported in every request so a bot can budget its computation.
- **Resources:** 1 vCPU, 1 GB RAM, no network, read-only filesystem.

9 Errors and Corrections

Situation	Result
Illegal action, or a reply that cannot be parsed (not JSON, missing fields, unknown card)	Corrected to **check** if legal, otherwise **fold**. A `notice` is sent.
Illegal discard, or `card` missing or unparsable	The lowest-ranked of your 3 hole cards is discarded (2 lowest, ace highest). Among cards of equal rank, the one whose suit comes first in the order spades, hearts, diamonds, clubs is discarded. A `notice` is sent. No reply at all is a timeout.
Crash, or end of file on stdout	Forfeit of the remaining match: a loss with $\Delta = -C$, and the opponent's result is set to $\Delta = +C$. A forfeit in either leg forfeits the whole match; after a first-leg forfeit the second leg is not played.
Time bank exhausted, reply later than 5 seconds, or wall-clock kill	Same as a crash.
Reply when no request is outstanding	Ignored.

10 Example Hand

The bot is on the button in hand 1 of a Stage 1 leg. Messages are shown from the bot's point of view; `E>B` is engine to bot, `B>E` is bot to engine. Lines are shortened to fit; `...` marks history entries left out for readability (real messages contain every entry). Opponent replies are not shown, because the bot only sees their effect.

```
E>B {"type":"init","protocol_version":"1.0","total_hands":200,"starting_stack":200,"small_blind
    ":1,"big_blind":2,"time_bank_ms":30000,"per_response_ms":5000}
E>B {"type":"new_hand","hand_id":1,"seat":"button","hole_cards":["As","Kd","7c"],"stack":200,"
    opp_stack":200}
E>B {"type":"action_request","hand_id":1,"street":"preflop","board":[],"pot":3,"my_bet":1,"
    opp_bet":2,"to_call":1,"my_stack":199,"opp_stack":198,"legal_actions":["fold","call","raise
    "],"min_raise":4,"max_raise":200,"raises_left":3,"history":[],"time_left_ms":30000}
B>E {"type":"action","action":"raise","amount":6}
   (opponent calls, the flop is dealt, opponent checks)
E>B {"type":"action_request","hand_id":1,"street":"flop","board":["Ah","7d","2s"],"pot":12,"
    my_bet":0,"opp_bet":0,"to_call":0,"my_stack":194,"opp_stack":194,"legal_actions":["check","
    raise"],"min_raise":2,"max_raise":194,"raises_left":4,"history":[{"street":"preflop","actor
    ":"me","action":"raise","amount":6},{"street":"preflop","actor":"opp","action":"call","
    amount":6},{"street":"flop","actor":"opp","action":"check"}],"time_left_ms":29940}
B>E {"type":"action","action":"raise","amount":6}
   (opponent calls; flop betting is over)
E>B {"type":"discard_request","hand_id":1,"hole_cards":["As","Kd","7c"],"board":["Ah","7d","2s
    "],"pot":24,"my_stack":188,"opp_stack":188,"time_left_ms":29900}
B>E {"type":"discard","card":"Kd"}
E>B {"type":"discard_result","hand_id":1,"my_discard":"Kd","opp_discard":"Qs","hole_cards":["As
    ","7c"]}
   (turn card 3c is dealt, opponent checks)
E>B {"type":"action_request","hand_id":1,"street":"turn","board":["Ah","7d","2s","3c"],"pot
    ":24,"my_bet":0,"opp_bet":0,"to_call":0,"my_stack":188,"opp_stack":188,"legal_actions":["
    check","raise"],"min_raise":2,"max_raise":188,"raises_left":4,"history":[ ... {"street":"
    turn","actor":"opp","action":"check"}],"time_left_ms":29860}
B>E {"type":"action","action":"check"}
   (river card 4h is dealt, opponent checks)
E>B {"type":"action_request","hand_id":1,"street":"river","board":["Ah","7d","2s","3c","4h"],"
    pot":24,"my_bet":0,"opp_bet":0,"to_call":0,"my_stack":188,"opp_stack":188,"legal_actions
```

```
    ":["check","raise"],"min_raise":2,"max_raise":188,"raises_left":4,"history":[ ... {"street
    ":"river","actor":"opp","action":"check"}],"time_left_ms":29820}
B>E {"type":"action","action":"raise","amount":10}
   (opponent folds)
E>B {"type":"hand_result","hand_id":1,"result":"fold","net":12,"board":["Ah","7d","2s","3c","4h
    "],"my_hole_cards":["As","7c"],"opp_hole_cards":null,"opp_discard":"Qs","history":[ ... ]}
   ... hands 2 to 200 ...
E>B {"type":"match_end","reason":"complete"}
   (the bot exits; the second leg starts a new bot process with its own init)
```

Chip check for hand 1: the bot committed $6 + 6 + 10 = 22$, the opponent 12; the opponent folded, so the bot won the opponent's 12 chips (`net` $= +12$).

11 Starter Bot (Python 3.11, standard library only)

```python
import sys, json

RANKS = "23456789TJQKA"

def choose_action(req):
    legal = req["legal_actions"]
    # Replace with your strategy. This one checks if free, else calls.
    if "check" in legal:
        return {"type": "action", "action": "check"}
    if "call" in legal:
        return {"type": "action", "action": "call"}
    return {"type": "action", "action": "fold"}

def choose_discard(req):
    # Replace with your strategy. This one discards the lowest rank.
    cards = req["hole_cards"]
    worst = min(cards, key=lambda c: RANKS.index(c[0]))
    return {"type": "discard", "card": worst}

def main():
    for line in sys.stdin:
        line = line.strip()
        if not line:
            continue
        try:
            msg = json.loads(line)
        except ValueError:
            continue
        t = msg.get("type")
        if t == "action_request":
            reply = choose_action(msg)
        elif t == "discard_request":
            reply = choose_discard(msg)
        elif t == "match_end":
            break
        else:
            # init, new_hand, discard_result, hand_result, notice:
            # update your own state here (it lasts for this leg only); no reply is sent.
            continue
        print(json.dumps(reply), flush=True)

if __name__ == "__main__":
    main()
```

12 Checklist Before You Upload

- The bot runs with `python3.11 bot.py` and uses only the standard library, numpy and scipy.
- Nothing but protocol JSON is printed to stdout, and every reply is flushed.
- The bot answers both `action_request` and `discard_request`.
- The bot ignores message types it does not know.
- The bot exits after `match_end`.
- The bot has been tested against the starter kit's local match runner for a full Stage 1 match (two legs of 200 hands) inside the per-leg time bank.